

Assignment - 4

(User Interface for ML models)

Name - Aakanksha Kumari
Roll no. - 210006

Name - Manish Kumar
Roll no. - 251330604

Working link: <https://cs-698-hai-project-nkptrb4d5-manishs-projects-b5e8b6ac.vercel.app/>
GitHub Repository: <https://github.com/manishkr21/CS698-HAI-Project>

1. Intended Users and System Purpose

The Student Dropout Analysis Platform is designed for university administrators, academic advisors, and education policymakers.

These users need to identify at-risk students early and ensure that predictive models remain fair, transparent, and interpretable across demographic and socioeconomic groups.

The system connects the machine learning models with a user-friendly front-end interface that:

- Allows users to input student details.
- Generates predictions with confidence levels.
- Displays overall model performance.
- Highlights fairness metrics before and after bias mitigation.
- Clearly explains what the model can and cannot do.

The interface ensures that predictions are used responsibly, emphasizing that outputs support academic interventions rather than punitive decisions.

2. Interface Design and User Flow

The interface was developed using a web-based dashboard architecture (React + FAST API). The design aligns with user goals: prediction, model comparison, and fairness interpretation.

a) Predict Tab

The Make Prediction section allows users to enter student attributes:

- Academic: Admission Grade, Course, Number of Units Enrolled/Approved.
- Demographic: Age, Gender, Attendance Type.
- Socio-economic: Debtor Status, Scholarship Holder, Special Needs, International Status.

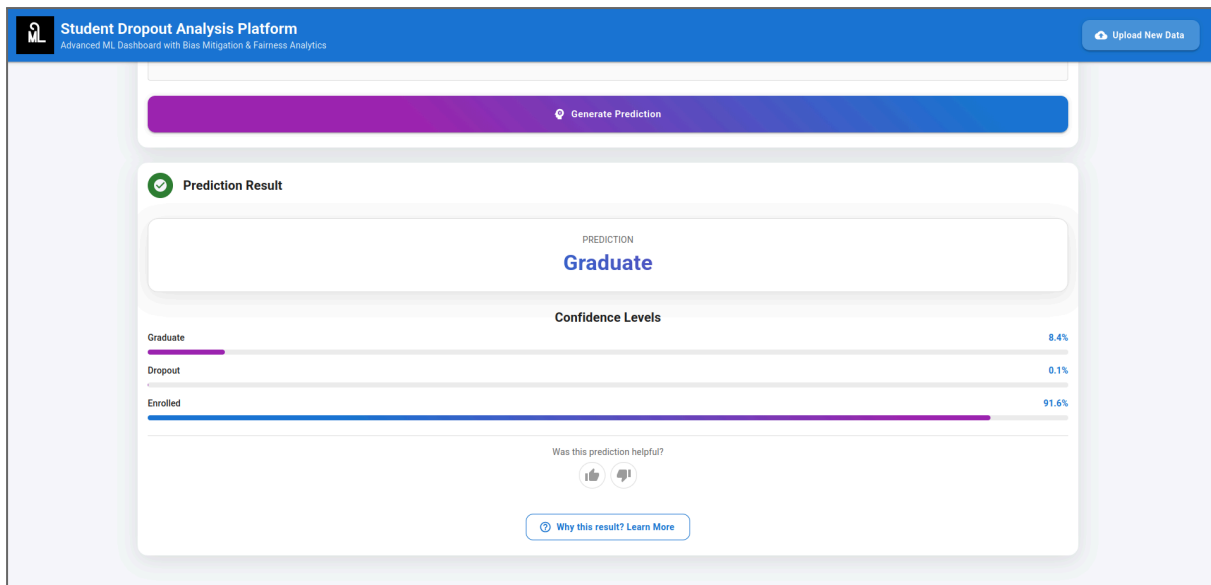
The screenshot displays the 'Student Dropout Analysis Platform' interface. The header includes the platform name and a tagline 'Advanced ML Dashboard with Bias Mitigation & Fairness Analytics', along with an 'Upload New Data' button. The main form is titled 'Try with Example' and contains several input sections:

- Demographic:** 'Admission grade' (120, range 0-200), 'Gender' (Male), and 'Age at enrollment' (20, range 17-70).
- Consent & Status:** A row of six status fields, each with 'Yes' and 'No' radio buttons:
 - Displaced: No (selected)
 - Educational special needs: No (selected)
 - Debtor: No (selected)
 - Tuition fees up to date: Yes (selected)
 - Scholarship holder: No (selected)
 - International: No (selected)
- Academic Performance:** Two columns for semester data:
 - First Semester:** 'Curricular units 1st sem (credited)' (0) and 'Curricular units 1st sem (enrolled)' (6).
 - Second Semester:** 'Curricular units 2nd sem (credited)' (0) and 'Curricular units 2nd sem (enrolled)' (6).

A large blue button at the bottom is labeled 'Generate Prediction'.

After clicking "Generate Prediction", the system displays:

- The predicted outcome (*Graduate*, *Dropout*, or *Enrolled*).
- Confidence probabilities for all three classes.
- Comparison between baseline and bias-mitigated models.
Visual indicators like colored labels ("Predictions Match" / "Predictions Differ") communicate whether fairness interventions affected results.



b) Performance & Fairness Tab

Model Performance Table

Shows Validation Accuracy, Precision, Recall, and F1-Score for all three models (Logistic Regression, Random Forest, Gradient Boosting).

Below the table, a "Key Insights & Analysis" panel summarizes:

- Which model performs best?
- Trade-offs between interpretability and accuracy.
- How bias mitigation changed performance.

Performance Bar Chart

Displays metrics visually for easy comparison across models.

Administrators can instantly identify which model generalizes best and where fairness corrections affected performance.

Fairness Indicators

The system shows:

- Group-wise fairness metrics (Statistical Parity Difference, Disparate Impact, Equal Opportunity Difference).
- Comparison before and after mitigation.

Clear textual insights explain:

"The corrected model is slightly less accurate but significantly fairer."

This ensures that users understand that fairness improvement can come with minor accuracy trade-offs.

c) Help Tab

The landing page provides a quick-start guide describing:

- What the model does.
- What features are required for prediction?
- Its limitations and fairness considerations.
Sections like "*Getting Started*" and "*Model Performance*" summarize the model's scope, ensuring transparency for non-technical users.

3. Communicating Model Capabilities and Limitations

To promote responsible AI use, the interface communicates both capabilities and boundaries.

Model Capabilities

- Predicts dropout, enrollment, or graduation status.
- Shows prediction confidence for each class.
- Compares model performances and fairness metrics.
- Explains fairness trade-offs and bias mitigation impact.

Model Limitations

- Predictions are probabilistic—not definitive judgments.
- The model does not explain personal causes of dropout.
- Results rely on past data; new patterns may affect accuracy.
- Sensitive attributes (e.g., nationality) are excluded to prevent discrimination.

The Help and Insights sections include these disclaimers, promoting ethical and transparent deployment.

4. HAX-Based Evaluation

The interface was evaluated using the Human-AI eXperience (HAX) principles.

S.no	HAX Guideline	How the Interface Meets It	Improvement / Future Work
1.	Make clear what the system can do	The "Help" tabs describe the possible FAQ.	Add a short guided walkthrough when the user

			first visits the app to introduce all features.
2.	Make clear how well the system can do what it can do	Performance metrics (Accuracy, Precision, Recall, F1-score) are clearly displayed in tables and bar charts to show reliability.	Add visual indicators (e.g., confidence bands) or statements about uncertainty ranges for each prediction.
3.	Time services based on context	The system only provides predictions after the user submits all inputs—avoiding untimely alerts or pop-ups.	Add timed tips or feedback only when users pause or make input mistakes (contextual help).
4.	Show contextually relevant information	Displays relevant details—model confidence, fairness metrics, and performance—based on the user’s current tab.	Add adaptive tooltips that explain metrics when the cursor hovers over them.
5.	Match relevant social norms	The language is neutral, formal, and aligned with the academic context (used by administrators and educators).	Localize interface text for multilingual audiences or culturally diverse institutions.
6.	Mitigate social biases	Bias-mitigation models are built into the backend, and fairness visualizations explicitly compare baseline vs. mitigated results.	Expand fairness assessment to include intersectional bias (e.g., gender × financial aid).
7.	Support efficient invocation	Prediction generation is one-click (“Generate Prediction”) with clearly labeled input fields and buttons.	Allow bulk upload (CSV) for multiple student predictions in institutional use cases.
8.	Support efficient dismissal	Users can easily clear inputs or switch tabs without being interrupted by model suggestions.	Add a “Reset All Inputs” button for faster clearing.
9.	Support efficient correction	Users can instantly edit input values and regenerate predictions without restarting the session.	Include real-time input validation (e.g., “Age must be between 15–40”).
10.	Scope services when in doubt	If inputs are incomplete, no predictions are generated—ensuring safety and reliability.	Add uncertainty flags (“Low confidence prediction”) when probability differences are small.
11.	Make clear why the system did what it did	Each prediction displays class probabilities; fairness and feature importance are discussed in textual summaries.	Integrate SHAP or feature-importance charts to show which factors influenced predictions.

12.	Remember recent interactions	The interface does not design to save user information.	Add persistent user profiles or session history for comparison across multiple logins.
13.	Learn from user behavior	Not currently adaptive but could use user choices (e.g., preferred model) for personalization.	Implement learning from repeated administrator selections to improve recommendations.
14.	Update and adapt cautiously	The model updates only after explicit retraining; no automatic changes occur during use.	Add clear "Model updated" notifications when backend versions are refreshed.
15.	Encourage granular feedback	After each prediction it ask for user feedback.	Feed may be use to improve the model
16.	Convey the consequences of user actions	Textual summaries explain that predictions are advisory and should not be used punitively.	Add confirmation prompts when users download reports or act on recommendations.
17.	Provide global controls	Admins can switch between baseline and bias-mitigated models globally from the comparison dashboard.	Add confirmation prompts when users download reports or act on recommendations.
18.	Notify users about changes	The interface shows when models or metrics are refreshed (e.g., "Bias Mitigation Applied").	Add subtle in-app notifications (e.g., toast alerts) whenever model data updates occur.

The interface strongly fulfills Explainability, Fairness Awareness, and Transparency, with room for expansion in interactive controls and personalized explanations.